Our original architecture centered around using softcore RISC-V targets on an FPGA, paired with a glitcher and a controller. This design allowed deep control over the processor, fault injection at the ISA level, clock tree manipulation, and testing specific low-level behaviors. While powerful, it also had limitations, especially when it came to simulating complete and real-world systems. The new direction involves replacing the softcore RISC-V with a fixed SoC, for example, the NXP i.MX8M Plus or the NVIDIA Jetson offers a more practical path that aligns better with actual automotive and Edge-AI environments.

With this change, we do lose access to the processor's internals. We can no longer inject faults directly into the clock, modify the ISA, or interact with the CPU pipeline, which were the advantages of the FPGA flow, but also made testing abstract and somewhat disconnected from how real systems behave under fault. Now, instead of controlling the CPU from the ground up, we treat it as a black box and focus on its inputs, outputs, and behavior under stress.

What we gain, however, is speed, realism, and relevance. On the SoC, we can run full-systems with LiDAR, camera input, and neural networks, all in an actual operating system, which means we no longer simulate functionality; we test actual deployed software. The Jetson and i.MX8M platforms support Linux, hardware acceleration, and sensor interfaces, which allows us to prototype applications quickly. Instead of synthesizing HDL every time, we can write Python or C++, test performance directly, and move much faster.

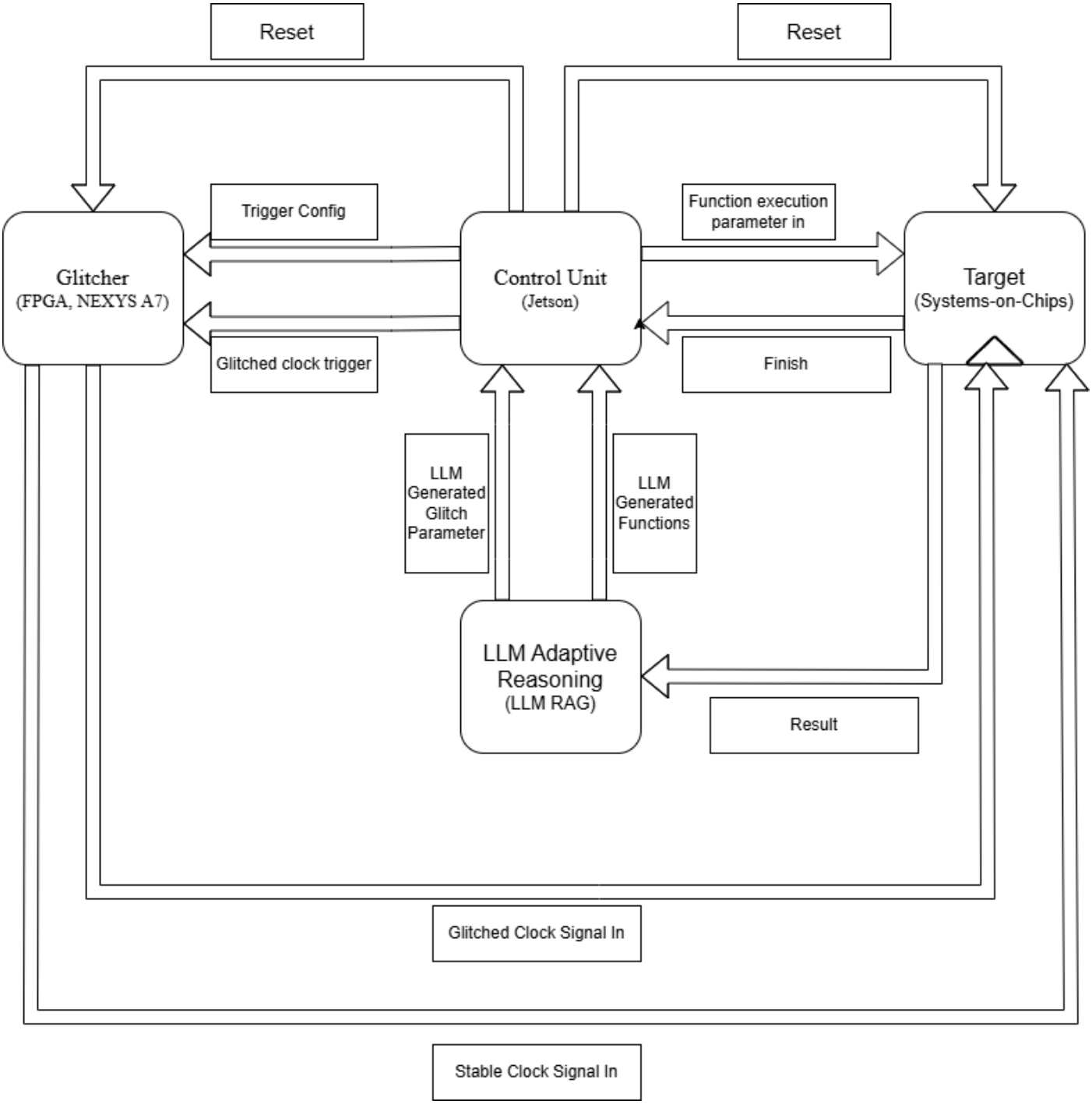
Another advantage is power monitoring. For example, we can monitor CPU and GPU power usage real-time, directly from the terminal on Jetson Orin boards. We no longer need separate ADCs or power sensors to observe how a glitch affects power consumption. If a fault causes a spike or crash, we can catch it instantly and match it with software logs.

Our glitcher, still running on the Nexys A7, remains responsible for injecting physical faults. It connects to the target SoC through GPIO. The controller board, whether it is a Jetson Nano, Raspberry Pi, or something else, sends commands to the glitcher, times the injections, and communicates with the target. The three boards communicate through GPIO, UART, I2C, or SPI, depending on available ports on the boards. If the controller only supports GPIO, we can keep it simple and drive signals directly, just like in basic robotics projects.

We inject faults by physically disturbing the lines going to sensors or power. For example, we can briefly disconnect or glitch a LiDAR input, interrupt a power line, or send corrupted data into the I2C bus. These faults are common in edge-AI systems exposed to harsh environments. They simulate real-world failures, dust, power drops, or EMI, not just theoretical bugs. This realism gives our work industrial relevance and practical impact.

To inject at the right moment, we can instrument the target software to send a signal to the controller when it's ready. The controller, in turn, activates the glitcher, making our setup precise without requiring complex protocols or timing mechanisms. It's faster to deploy, more reliable, and repeatable. We stick to our already set work-flow diagram, but our target changes to a SoC and subsequent deployments following that become easier.

In conclusion, we trade internal control for system realism. We no longer attack the CPU directly, but we test complete systems under realistic fault conditions. We move faster, deploy faster, and gather more actionable data. Given the updated project scope, this path makes sense and brings us closer to real-world fault resilience testing.


Reset
Reset
Trigger Config
Function execution parameter in
Glitcher
(FPGA, NEXYS A7)
Control Unit
(Jetson)
Target
(Systems-on-Chips)
Glitched clock trigger
Finish
LLM Generated Glitch Parameter
LLM Generated Functions
LLM Adaptive Reasoning
(LLM RAG)
Result
Glitched Clock Signal In
Stable Clock Signal In